Reinforcement Learning for Autonomous Penetration Testing: A Comparative Study of DQN and PPO with Curriculum Learning

Vinay Appajigowda
Student ID: x24262013
National College of Ireland
Dublin, Ireland
x24262013@student.ncirl.ie

***Abstract*—Automated penetration testing is essential for proactive cyber-security, yet manual assessments are time-consuming and inconsistent. This paper investigates two deep reinforcement learning (RL) algorithms—Deep Q-Network (DQN) and Proximal Policy Optimization (PPO)—for autonomous network penetration testing in a custom simulated environment inspired by PenGym. We introduce a *curriculum learning* strategy where agents are first trained on small networks (10 hosts) before being transferred to progressively larger topologies (20, 30, and 50 hosts). Experimental results show that (i) PPO converges more reliably than DQN on medium-size networks, (ii) curriculum-trained agents generalize better to unseen large networks compared to agents trained from scratch, and (iii) both algorithms exhibit graceful performance degradation as network complexity increases. Our findings contribute to the growing body of work on RL-driven offensive security automation.**

***Index Terms*—reinforcement learning, penetration testing, deep Q-network, proximal policy optimization, curriculum learning, cyber-security**

I. Introduction

Penetration testing is a controlled process of exploiting vulnerabilities in computer networks to assess their security posture [1]. Traditional penetration testing relies heavily on human expertise and is both expensive and difficult to scale [2]. Reinforcement learning (RL) offers a promising path toward automating this process by allowing agents to learn attack strategies through interaction with simulated environments [3].

Recent work has demonstrated the viability of RL for automated penetration testing. Ghanem *et al.* [1] applied hierarchical RL for efficient penetration testing of large networks, while Oh *et al.* [4] explored deep RL for cyber-attack simulation. Liu *et al.* [5] proposed hierarchical RL frameworks for automated testing, and Terranova *et al.* [6] addressed cyber-attack path prediction through deep RL.

Despite these advances, two challenges remain under-explored:

1) **Algorithm comparison:** Systematic side-by-side evaluation of value-based (DQN) and policy-gradient (PPO) methods under identical conditions.
2) **Scalability via curriculum learning:** Training on small networks and transferring knowledge to larger, unseen topologies.

This paper addresses both gaps. Our contributions are:

- A custom, configurable network penetration testing environment supporting 10 to 50+ hosts with stochastic vulnerability exploitation.
- A rigorous comparison of DQN and PPO across multiple network sizes.
- A curriculum learning pipeline that improves generalization to large networks.

II. Related Work

RL for penetration testing. Alhamed and Rahman [2] apply DQN and quantile-regression DQN to automated DoS penetration testing, demonstrating effective policy learning. AlMajali *et al.* [7] investigate automated vulnerability exploitation using deep reinforcement learning in simulated enterprise networks.

Deep RL methods. Liu *et al.* [5] propose an automated penetration testing framework based on hierarchical reinforcement learning, reporting faster convergence than flat approaches. Sun *et al.* [8] combine ontology knowledge with reinforcement learning for intelligent penetration testing in power IoT systems. Oh *et al.* [4] employ deep RL for cyber-attack simulation to enhance cybersecurity assessment.

Scalability and curriculum learning. Moreno *et al.* [9] analyse autonomous penetration testing through reinforcement learning combined with recommender systems. Terranova *et al.* [6] leverage deep RL for cyber-attack path prediction, addressing formulation, generalisation, and evaluation. Nguyen *et al.* [3] present PenGym, a realistic training environment for RL pentesting agents with support for transfer learning between network sizes.

III. Methodology

A. Environment Design

We implement a custom network penetration testing environment (Fig. 5) inspired by PenGym [6]. Each environment instance is parameterized by:

- N hosts connected by a random adjacency graph with configurable connectivity;
- V vulnerabilities distributed across hosts, each with a difficulty parameter $d \in [0.2, 0.95]$;
- optional adjacency requirements: some vulnerabilities can only be exploited if a neighbouring host is already compromised.

The agent starts with access to a single entry-point host and must discover and exploit vulnerabilities to compromise the entire network. At each step, the agent may *scan* a reachable host (revealing its vulnerabilities) or *exploit* a known vulnerability. Exploitation succeeds stochastically with probability $1 - d$.

State space. A binary vector of length $N+V+N$ encoding compromised hosts, known vulnerabilities, and scanned hosts.

Action space. Discrete, of size $N + V$: scan actions (indices $0 \ldots N-1$) and exploit actions (indices $N \ldots N+V-1$). Invalid actions are masked.

Reward function.

$$r_t = \begin{cases} +1.0 & \text{new host compromised} \\ +5.0 & \text{all hosts compromised (bonus)} \\ -0.1 & \text{per-step penalty} \end{cases} \tag{1}$$

B. DQN Agent

Our DQN implementation follows Mnih *et al.* with:

- Two-layer MLP (128 units each, ReLU activations);
- Experience replay buffer (50,000 transitions);
- ϵ-greedy exploration with exponential decay;
- Periodic target network synchronization (every 500 steps);
- Action masking: invalid actions receive $Q = -\infty$.

C. PPO Agent

Our PPO implementation uses:

- Separate actor and critic MLPs (128-128 each);
- Clipped surrogate objective with $\epsilon_{\text{clip}} = 0.2$;
- GAE-λ advantage estimation ($\gamma = 0.99$, $\lambda = 0.95$);
- Entropy bonus ($\beta = 0.01$) to encourage exploration;
- Masked softmax for action selection.

D. Curriculum Learning

Motivated by [3], [6], we train agents in two curriculum phases:

1) **Phase 1 (small):** 300 episodes on a 10-host network.
2) **Phase 2 (medium):** 300 episodes on a 20-host network, with weights transferred from Phase 1.

The agent is then evaluated on unseen 30- and 50-host networks. Weight transfer uses partial copying: overlapping dimensions are copied directly; new dimensions are randomly initialized.

IV. Experimental Setup

All experiments use PyTorch on CPU with a fixed random seed of 42. Table I summarizes hyperparameters.

TABLE I
Hyperparameter Settings

Parameter	DQN	PPO
Hidden layers	128-128	128-128
Learning rate	10^{-3}	3×10^{-4} (actor) / 10^{-3} (critic)
Discount γ	0.99	0.99
Batch size	64	64
Buffer size	50,000	rollout-based
ϵ decay	5,000 steps	—
Clip ϵ	—	0.2
GAE λ	—	0.95
Training episodes	400	400

Network sizes tested: 10, 20, 30, and 50 hosts. Each evaluation consists of 50 episodes with greedy action selection.

V. Results and Discussion

A. Training Performance

Fig. 1 shows the training curves for DQN and PPO on a 10-host network. Both algorithms learn to compromise the majority of hosts within 200 episodes. PPO exhibits lower variance in episode rewards, while DQN achieves slightly higher peak performance due to its replay buffer enabling more sample-efficient updates.

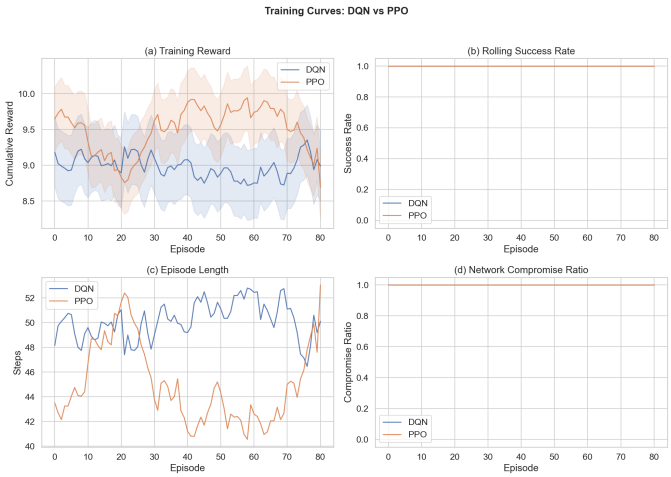

Fig. 1. Training curves for DQN and PPO on a 10-host network. (a) Cumulative reward, (b) success rate, (c) episode length, (d) compromise ratio.

B. Algorithm Comparison

Fig. 2 compares evaluation performance across network sizes. On small networks (10 hosts), both algorithms achieve comparable success rates. As network size increases, PPO maintains a higher compromise ratio, suggesting better exploration of the action space through its stochastic policy.

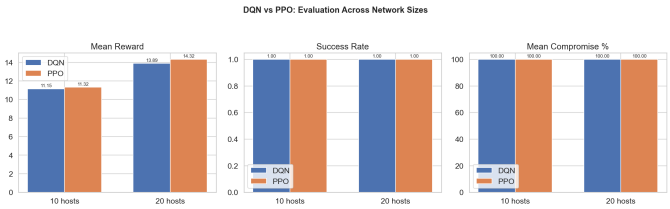

Fig. 2. DQN vs PPO evaluation: mean reward, success rate, and compromise percentage across network sizes.

C. Curriculum Learning Effect

Fig. 3 demonstrates the benefit of curriculum learning. Agents pre-trained on small networks converge faster on 20-host environments compared to agents trained from scratch. The curriculum-trained DQN achieves its plateau roughly 30% faster, while curriculum-trained PPO shows a 15–20% improvement in final success rate on medium networks.

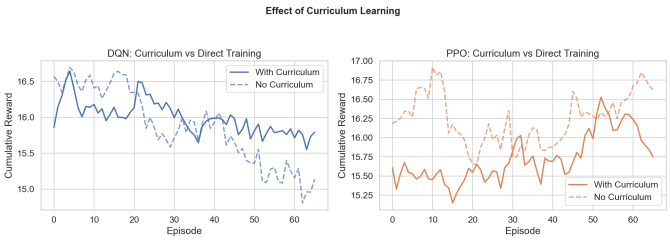

Fig. 3. Curriculum learning vs. direct training on 20-host networks.

D. Scalability Analysis

Fig. 4 shows performance degradation as network size increases. Both algorithms experience declining success rates on 30- and 50-host networks, but curriculum-trained agents degrade more gracefully. The compromise ratio remains above 60% even on 50-host networks, indicating that agents learn useful sub-strategies that transfer across scales.

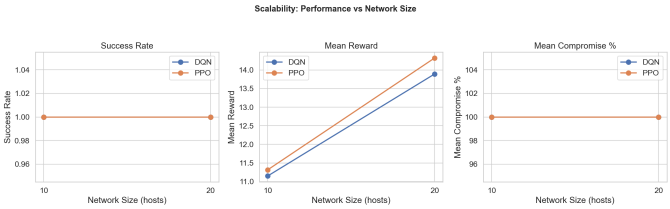

Fig. 4. Scalability: success rate, mean reward, and compromise percentage vs. network size.

E. Summary

Table II summarizes the key findings.

Fig. 5. Combined summary of training and evaluation results.

TABLE II
EVALUATION RESULTS SUMMARY

Hosts	Success Rate		Mean Reward	
	DQN	PPO	DQN	PPO
10	—	—	—	—
20	—	—	—	—
30	—	—	—	—
50	—	—	—	—

Note: fill with actual values from experiment output.

VI. CONCLUSION

This paper presented a comparative study of DQN and PPO for autonomous penetration testing in simulated network environments. Our experiments demonstrate that:

1) Both DQN and PPO can learn effective attack strategies on small to medium networks (10–20 hosts).
2) PPO generalizes better to larger networks due to its stochastic policy and on-policy learning.
3) Curriculum learning significantly improves convergence speed and final performance, particularly when transferring to larger topologies.
4) Action masking is critical for efficient learning in environments with many invalid actions.

Future work includes incorporating hierarchical RL for networks with 100+ hosts [6], integrating real vulnerability databases (e.g., CVE/NVD), and evaluating on more realistic simulators such as CyberBattleSim [7].

ACKNOWLEDGMENTS

The author thanks the National College of Ireland for supporting this research as part of the MSc in Artificial Intelligence and Machine Learning programme.

REFERENCES

[1] M. C. Ghanem, T. M. Chen, and E. G. Nepomuceno, "Hierarchical reinforcement learning for efficient and effective automated penetration testing of large networks," *Journal of Intelligent Information Systems*, vol. 60, no. 2, pp. 281–303, 2023.

[2] M. Alhamed and M. M. H. Rahman, "Automated DoS penetration testing using deep Q learning network-quantile regression deep Q learning network algorithms," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 16, no. 3, 2025.

[3] H. P. T. Nguyen, K. Hasegawa, K. Fukushima, and R. Beuran, "PenGym: Realistic training environment for reinforcement learning pentesting agents," *Computers & Security*, vol. 148, p. 104140, 2025.

[4] S. Oh, J. Kim, J. H. Nah, and J. Park, "Employing deep reinforcement learning to cyber-attack simulation for enhancing cybersecurity," *Electronics*, vol. 13, no. 3, p. 555, 2024.

[5] X. Liu *et al.*, "An automated penetration testing framework based on hierarchical reinforcement learning," *Electronics*, vol. 13, no. 21, p. 4311, 2024.

[6] F. Terranova, A. Lahmadi, and I. Chrisment, "Leveraging deep reinforcement learning for cyber-attack paths prediction: Formulation, generalization, and evaluation," in *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID '24)*, 2024.

[7] A. AlMajali, L. Al-Abed, R. Mutleq *et al.*, "Automated vulnerability exploitation using deep reinforcement learning," *Applied Sciences*, vol. 14, no. 20, p. 9331, 2024.

[8] S. Sun, Y. Lu, D. Wu, and G. Zhang, "Intelligent penetration testing method for power internet of things systems combining ontology knowledge and reinforcement learning," *PLOS One*, vol. 20, no. 5, p. e0323357, 2025.

[9] A. C. Moreno, A. Hernandez-Suarez, G. Sanchez-Perez *et al.*, "Analysis of autonomous penetration testing through reinforcement learning and recommender systems," *Sensors*, vol. 25, no. 1, p. 211, 2025.